GUI Engine

Die AGATE GUI Engine automatisiert Weboberflächen mit **Playwright für Java**. Navigation, Benutzeraktionen und Prüfungen werden als YAML beschrieben. Diese Seite erklärt den Einstieg, die verfügbaren Elemente, die Browserkonfiguration und das Verhalten von Browser-Sessions. Am Ende stehen die Ergebnisse eines Browservergleichs und die noch offenen technischen Punkte.

Geltungsbereich: Die Beispiele beziehen sich auf die beschriebene AGATE-Implementierung und die lokale `agate-learning-gui-demo`. Ob eine Funktion verfügbar ist, hängt vom tatsächlich gebauten und gestarteten `agate-server` ab. Die Version 1.0.75 vereinfacht das Logging; sie behebt **nicht** das zuvor beobachtete Problem mit zurückbleibenden Browserprozessen.

1. Browser starten und Seite öffnen

Der Browser wird beim ersten GUI-Zugriff automatisch gestartet. Ein eigener Browser-Start-Schritt ist nicht erforderlich. `OPEN` erstellt bei Bedarf die GUI-Session und navigiert zur angegebenen URL.

```yaml
name: GUI – Einstieg
timeout: 10000
steps:
  - section: Browser starten
    description: Die lokale Lernseite öffnen.
  - id: open-learning-page
    type: GUI
    op: EXEC
    action: OPEN
    page: Lernübersicht
    url: 'http://127.0.0.1:8767/'
  - id: learning-page-visible
    type: GUI
    op: ASSERT
    action: VISIBLE
    target:
      css: '#page-HOME'

```

Voraussetzung: Der lokale Demo-Service muss bereits laufen (`agate-learning-gui-demo/start-demo.bat`). Der GUI-Schritt startet den Browser, **nicht** den Demo-Webservice.

`page` ist eine frei wählbare Beschreibung für das Log; sie ist weder URL noch Locator und wählt keinen Browser-Tab aus. Eine bereits geöffnete persönliche Edge- oder Chrome-Session wird **nicht** übernommen.

2. Browser auswählen und konfigurieren

2.1 Unterstützte Browser

YAML-Wert	Gestarteter Browser	Voraussetzung
`chromium`	Von Playwright verwalteter Chromium	Passende Browser-Binaries zur Playwright-Version
`edge`	Installierter Microsoft Edge (`msedge`-Kanal)	Edge ist auf dem Rechner installiert
`chrome`	Installierter Google Chrome (`chrome`-Kanal)	Chrome ist auf dem Rechner installiert

Ohne explizite Auswahl gilt `chromium`. Die Aliase `internal` für `chromium` und `msedge` für `edge` werden akzeptiert. Ungültige Werte werden abgewiesen. Ein fehlgeschlagener Edge- oder Chrome-Start führt **nicht** automatisch zum Wechsel auf einen anderen Browser.

2.2 Empfohlene Konfiguration auf Suite-Ebene

```yaml
gui:
  browser: edge
  headless: true
  reuse: true

testCases:
  - id: GUI_001
    description: Startseite prüfen
    timeout: 30000
    steps:
      - id: open-home
        type: GUI
        op: EXEC
        action: OPEN
        url: 'http://127.0.0.1:8767/'

```

Parameter	Werte	Standard ohne ausdrückliche Konfiguration	Wirkung
`browser`	`chromium`, `edge`, `chrome`	`chromium`	Browser-Engine bzw. installierter Browserkanal
`headless`	`true` / `false`	`false`	Ohne bzw. mit sichtbarem Browserfenster
`reuse`	`true` / `false`	`true`	Browserprozess innerhalb **einer Suite** wiederverwenden oder für jeden

			TestCase separat starten
`lifecycleLog`	`true` / `false`	`false`	Ausführliche Diagnose des Browser-Lebenszyklus

Die Parameter sind optional. Für die tägliche Arbeit reicht beispielsweise:

```
gui:
  browser: chrome
  headless: true
  reuse: true

```

Kein `sessionMode` in neuen Beispielen: AGATE unterstützt derzeit ausschließlich isolierte Browserkontexte (`ISOLATED`). Deshalb wird der Parameter weder benötigt noch in den Demo-Testfällen gezeigt. Bestehende YAML-Dateien mit `sessionMode: ISOLATED` bleiben kompatibel; `SHARED` ist nicht unterstützt.

2.3 Alternative Konfiguration beim Start

Der Browser kann auch außerhalb der Suite gewählt werden:

```
REM Vor dem Java-Aufruf in startTests.bat:
set "AGATE_GUI_BROWSER=edge"

```

Oder über JVM-Systemproperties:

```
java -Dagate.gui.browser=edge -Dagate.gui.headless=true -jar
"%AGATE_JAR%" %*

```

Bei der Browserauswahl liest der beschriebene Code zunächst die JVM-Property `agate.gui.browser` , dann `AGATE_GUI_BROWSER` und verwendet ansonsten `chromium` . Ein in der YAML-Suite explizit gesetzter `gui.browser` -Wert bestimmt die Auswahl für diese Suite. Das tatsächlich gestartete JAR muss die Konfiguration unterstützen; die Batchdatei darf Werte nicht später ungewollt überschreiben.

Ein benutzerdefinierter `executable` -Pfad ist eine Sonderkonfiguration. Er kann erklären, warum trotz `browser: edge` eine andere ausführbare Datei gestartet wird.

3. Browser, BrowserContext und GUI-Session

Für die Testisolation sind **Browserprozess**, **BrowserContext** und **Page/Tab** zu unterscheiden:

- **Browserprozess:** Der gestartete Edge-, Chrome- oder Chromium-Prozess mit zugehörigen Unterprozessen.

- **BrowserContext:** Isolierter Arbeitsbereich mit eigener Cookie- und Web-Storage-Verwaltung.
- **Page/Tab:** Ein Browser-Tab innerhalb eines Contexts; bei weiteren Aktionen kann er weiterverwendet werden.

Innerhalb **eines TestCase** bleibt der aktive Context für aufeinanderfolgende GUI-Schritte erhalten. Cookies, `localStorage` und Seitenzustand können also bewusst zwischen Schritten genutzt werden. Am TestCase-Ende wird sein Context geschlossen. Ein neuer TestCase erhält **immer einen neuen isolierten Context**.

3.1 `reuse: false`

Pro TestCase wird ein eigener Browser gestartet und nach diesem TestCase wieder geschlossen. Bei drei GUI-TestCases ergibt das typischerweise drei Browserstarts und drei Contexts.

3.2 `reuse: true`

Ein Browserprozess wird innerhalb der Suite wiederverwendet. Die TestCases bekommen trotzdem jeweils ihren eigenen Context. Bei drei GUI-TestCases ergibt das typischerweise **einen Browserstart und drei Contexts**. Der gemeinsam genutzte Browser wird erst nach dem letzten TestCase geschlossen.

Wichtig: `reuse: true` bedeutet **Browser-Reuse, nicht Session-Sharing**. Cookies und `localStorage` dürfen nicht von TestCase 1 nach TestCase 2 übertragen werden. Das wurde mit `13-browser-isolation.yaml` durch die Werte `CLEAN`, `EMPTY`, `ONE` und `TWO` geprüft.

3.3 Referenzbeispiel: `13-browser-isolation.yaml`

Praxisbeispiel für Browser-Isolation und Browser-Reuse: `data/learningdemo/13-browser-isolation.yaml` (Kurzbezeichnung: `browser-isolation.yaml`). Diese ausführbare Demo-Suite ist auch die Grundlage für den Browservergleich in **Abschnitt 11**. Sie enthält drei TestCases:

1. `AGATE_L13_01_WRITE`**:** Prüft zunächst leere Cookies und `localStorage` und speichert anschließend den Marker `ONE`.
2. `AGATE_L13_02_ISOLATED`**:** Erwartet trotz des vorherigen TestCase einen sauberen Context, schreibt den Marker `TWO` und ein Memo.
3. `AGATE_L13_03_CLEAN_AGAIN`**:** Prüft erneut, dass Cookies, `localStorage` und Memo keine Daten der vorherigen TestCases enthalten.

Ausführung (im Verzeichnis `agate-server`, bei gestartetem lokalen Demo-Webservice):

```
startTests.bat Milenko ECS_SYST_SYST learningdemo 13-browser-isolation.yaml

```

Die Suite lässt sich mit `browser: edge`, `chrome` oder `chromium` sowie `headless: true/false` und `reuse: true/false` ausführen. Dafür werden nur die Werte im `gui`-Block der Suite angepasst; `sessionMode` **ist nicht erforderlich**, da die TestCase-Contexts weiterhin isoliert bleiben.

Jeder TestCase muss weiterhin **einzeln und innerhalb einer Suite** funktionieren. Separate AGATE-Ausführungen (etwa für SYST und AUW) verwalten ihre Browser unabhängig voneinander. Das löst allerdings keine Konflikte bei gemeinsamen externen Ressourcen wie Kartenlesern, Testkarten oder Datenbank-Testdaten.

4. Logausgabe und Lifecycle-Diagnose

4.1 Standardlog ab 1.0.75

Zu Beginn der Suite wird die wirksame Browserkonfiguration ausgegeben:

```
[GUI] CONFIG browser=edge headless=true reuse=false

```

Am Ende steht ein kompakter Überblick:

```
[GUI] SUMMARY browserLaunches=3 contexts=3 cleanupFailures=0
forcedDriverStops=3

```

Die konkrete Browser-Version bleibt bei Bedarf im normalen Ablauf erkennbar, z. B.:

```
[GUI browser] Installed Microsoft Edge 155.0.4283.45

```

`cleanupFailures=0` sagt nur, dass AGATE im eigenen Cleanup **keinen Fehler erkannt** hat. Es ist **kein Nachweis**, dass Windows keine Browser-Unterprozesse mehr enthält. `forcedDriverStops` zeigt, wie häufig der Playwright-Treiber notfalls beendet werden musste.

4.2 Detaillierte Diagnose nur bei Bedarf

```
gui:
  browser: edge
  headless: true
  reuse: false
  lifecycleLog: true

```

Bei `lifecycleLog: true` werden die technischen Ereignisse wieder ausgegeben, beispielsweise:

```
[GUI lifecycle] CONTEXT_CLOSE_BEGIN id=...
[GUI lifecycle] CONTEXT_CLOSE id=... durationMs=18
[GUI lifecycle] PLAYWRIGHT_CLOSE_BEGIN owner=... capturedChildren=6
[GUI lifecycle] DRIVER_FORCE_STOP owner=... graceMs=3000
[GUI lifecycle] OWNED_PROCESS_VERIFY owner=... driverAlive=false
childrenAlive=0 survivors=[]

```

Zusätzlich erscheinen detaillierte `[GUI metrics]`-Zeilen zu Startup, Cleanup und verbleibender Laufzeit. Dieselbe Diagnose kann per JVM-Property `-Dagate.gui.lifecycleLog=true` aktiviert werden. Ein explizites `lifecycleLog: false` in der YAML-Konfiguration hat Vorrang. **Fehler und Warnungen dürfen nicht durch den kompakten Logmodus verborgen werden.**

Das `verbose`-Argument eines `CALL`-Schritts ist davon unabhängig: Es steuert die Ausgabe der Reusable-Schritte, nicht den Playwright-Lifecycle.

5. Timeout steuern

Ein Timeout ist die **maximale** Wartezeit für eine GUI-Aktion oder Prüfung. Er verlängert einen erfolgreichen Schritt nicht künstlich und ist auch nicht die gesamte Zeitgrenze des TestCase.

Priorität	Konfiguration	Beispiel
1	`timeout` des einzelnen Schritts	`2500` ms
2	`timeout` des TestCase	`10000` ms
3	Fallback in `GuiStepConfig.resolve`	`30000` ms

```yaml
name: GUI – Timeout
timeout: 10000
steps:
  - type: GUI
    op: EXEC
    action: OPEN
    url: 'http://127.0.0.1:8767/01-links.html'
  - type: GUI
    op: ASSERT
    action: VISIBLE
    link:
      id: L01-help
  - type: GUI
    op: ASSERT
    action: VISIBLE
    timeout: 2500
    link:
      id: L01-help

```

Alle Werte sind Millisekunden und müssen positiv sein. Reusable-GUI-Schritte verwenden den Timeout des aufrufenden TestCase, sofern der jeweilige Schritt nichts anderes setzt. Ein `CALL`-Parameter namens `timeout` ist nicht automatisch eine GUI-Timeout-Konfiguration. Automatische Prüfungen warten auf den gewünschten Zustand; ein höherer Timeout repariert keinen falschen Locator.

Für Lernpausen, **nicht** als allgemeine Synchronisationsstrategie:

```yaml
- type: WAIT
  value: '5000'
```

```
3
```

6. Screenshots, Fehleranalyse und Trace

6.1 Screenshot bei Fehlern

Bei Fehlern innerhalb der GUI-Ausführung versucht AGATE automatisch einen Screenshot des aktuellen Viewports zu speichern:

```
output/gui/<Session-UUID>/step-<Nummer>-failed.png
```

Im Log erscheint der Pfad als `Evidence`. Der Screenshot-Versuch hat eine eigene Grenze von 5000 ms. Wenn noch keine gültige Seite vorhanden ist oder der Fehler schon vor der GUI-Ausführung entsteht, kann Evidence fehlen.

Ein anderer Basisordner lässt sich beim Start setzen:

```
java -Dagate.gui.artifacts=output/gui-evidence -jar "%AGATE_JAR%" %*
```

6.2 Screenshot bewusst erstellen

```
- type: GUI
  op: EXEC
  action: SCREENSHOT
  value: learning-overview.png
```

Diese Aktion erstellt eine **Full-Page-Aufnahme** im Session-Ausgabeverzeichnis und benötigt keinen Element-Locator.

6.3 Playwright Trace

```
java -Dagate.gui.trace=true -jar "%AGATE_JAR%" %*
```

Das Tracing ergänzt Screenshots und Snapshots. Beim Session-Cleanup wird `trace.zip` im Session-Verzeichnis abgelegt. Bei abruptem Prozessabbruch ist die Trace-Datei nicht garantiert.

7. Automatisierbare Hauptelemente

Die vereinfachten YAML-Elementformen werden durch `GuiElementTarget` umgesetzt. Pro Schritt wird eine vereinfachte Elementform **oder** ein technischer `target`-Locator angegeben.

Element	YAML	Auswahl	Typische Aktionen und Prüfungen
Link	`link`	`text`, `id`	`CLICK`, `HOVER`, `PRESS`, `EXTRACT`, Sichtbarkeit, Text und Anzahl; `DOWNLOAD`

			benötigt ein Downloadereignis, `DOWNLOAD_PDF` den passenden POST-PDF-Vertrag
Button	`button`	`text`, `id`	`CLICK`, `HOVER`, `PRESS`, `VISIBLE`, `HIDDEN`, `ENABLED`, `DISABLED`, `EXISTS`, `COUNT`
Eingabefeld	`textbox`	`label`, `placeholder`, `id`	`FILL`, `PRESS`, `EXTRACT`, `VALUE`, `VALUE_GLOB`, `VALUE_MATCHES`; sensible Werte als `sensitive` behandeln
Dropdown	`dropdown`	`label`, `id`	`SELECT` über Options-Label, Value oder Index; technisch auch `labelGlob`, `labelPattern`, Mehrfachauswahl
Checkbox	`checkbox`	`label`, `id`	`CHECK`, `UNCHECK`, `CHECKED`, `UNCHECKED`
Radiobutton	`radio`	`label`, `id`	`CHECK`, `CHECKED`, `UNCHECKED`
Text	`text`	`value`, `id`	`TEXT`, `TEXT_EQUALS`, `TEXT_CONTAINS`, `TEXT_GLOB`, `TEXT_MATCHES`, `COUNT`, `EXTRACT`
Überschrift	`heading`	`text`, `id`	`VISIBLE`, `EXISTS`, `COUNT`,

			Textprüfungen

Hinweise: `index` ist optional und **nullbasiert**. `exact` ist bei Text-/Label-/Placeholder-Auswahl optional (AGATE-Standard: `true`), darf bei `id` aber nicht gesetzt werden. Innerhalb einer vereinfachten Form genau **eine** Auswahlstrategie verwenden. `value` innerhalb von `text` beschreibt den **Locator-Text**, nicht den Inhalt eines Eingabefelds. Für beliebige JavaScript-Dropdowns existiert kein automatisch passender `select`-Vertrag.

8. Weitere Elemente und technische Locators

Anwendungsfall	Technischer Ansatz	Grenze / Hinweis
Tabellen und Zellen	`target.css` mit `tableCell` (`row`, `cell`, `last`, `rowCellsAll`)	Tabellenkoordinaten sind **einsbasiert**; kein vereinfachtes `table`-Feld
Iframes	`target.frames`	CSS-Selektoren von außen nach innen
Panels/Container	`target.scope`	Schränkt den Suchbereich ein
Dateiupload	`target` auf `input[type=file]` und `UPLOAD`	Dateipfade über `value`
Offenes Shadow DOM	Passende CSS-/Rollen-Locators	Geschlossenes Shadow DOM nicht direkt zugänglich
Contenteditable	Passender `target`-Locator mit `FILL`	tatsächliche Editierbarkeit vorausgesetzt
HTML-Dialoge	DOM-Locator auf `dialog` und enthaltene Buttons	keine nativen Betriebssystemdialoge
SVG, Canvas, Progress, Meter	DOM-Locators und Zustands-/Attributprüfungen	keine dedizierte Canvas-Pixelaktion
Browser-Tabs	`OPEN_TAB`, `FIND_TAB`, `CLOSE_TAB`, Tab-Buffer	Tabs sind Browserressourcen und keine HTML-Elemente

Die Engine automatisiert Weboberflächen, **nicht** beliebige Betriebssystemfenster. Bei komplexen Widgets müssen Locator und Aktion zur tatsächlichen DOM-Struktur passen.

9. Lesbare Tests und Best Practices

```yaml
- section: Links prüfen
  description: Unterschiedliche Locator-Strategien demonstrieren.
- type: CALL
  command: reusable.01-links
  parameters:
    verbose: 'true'

```

`section` ist ein informativer Abschnitt für die Logausgabe, kein zusätzlicher ausführbarer Statistikschritt. Er bleibt unabhängig von `verbose` sichtbar.

Für gut wartbare Testfälle empfehlen sich stabile IDs, aussagekräftige Testnamen und eine klare Trennung von **Vorbereitung**, **Aktion** und **Prüfung**. Nach einer Aktion sollte das fachlich relevante Ergebnis geprüft werden. Mehrdeutige Treffer werden mit `scope` eingegrenzt; feste Wartezeiten bleiben Lern- und Ausnahmefälle. Bei Fehlern zuerst YAML-Quellschritt, Locator, aktuelle URL und Evidence zusammen ansehen.

10. Geplante Detailseiten

Thema	Vorgesehener Inhalt	Status
Links	HTML, Accessible Name, `exact`, Linkaktionen und Prüfungen	Ausführliche Referenz bereits vorhanden
Buttons	Beschriftung, Click, Enabled/Disabled, Varianten	Zu ergänzen
Eingabefelder	Label, Placeholder, FILL/PRESS, sensible Daten	Zu ergänzen
Dropdowns	Label/Value/Index, Muster und Mehrfachauswahl	Zu ergänzen
Checkboxen und Radio	Idempotente Aktionen und Gruppen	Zu ergänzen
Tabellen	Koordinaten, Zeilenkriterien, verschachtelte Tabellen	Zu ergänzen
Textprüfungen	Normalisierung, NBSP, Glob und Regex	Zu ergänzen
Frames und Tabs	Suchkontext, Navigation und Lebensdauer	Zu ergänzen
Dateien und PDF	Upload, Download, Original-PDF und Tababschluss	Zu ergänzen

Diagnose	Timeout, Screenshots, Trace, Lifecycle und Debugging	Zu ergänzen

Die Übersicht ist ein Ausbauplan, keine Liste bereits vorhandener Confluence-Seiten.

---

11. Browservergleich – Benchmark vom 13.10.2026

11.1 Testaufbau

Für den Vergleich wurde immer dieselbe Suite `learningdemo/13-browser-isolation.yaml` mit drei TestCases verwendet. Sie prüft den Zustand von Cookies, `localStorage` und einem gespeicherten Memo. Alle 12 Konfigurationen wurden erfolgreich abgeschlossen (**3/3 PASSED je Konfiguration, zusammen 36/36**).

Variiert wurden ausschließlich `browser`, `headless` und `reuse`; die Context-Isolation war in allen Fällen aktiv. Gemessen wurden die **gesamte Suite-Laufzeit** und die **Anzahl neuer, nach Suite-Ende noch sichtbarer Browserprozesse**. Jeder Fall wurde einmal gemessen. Das ist ein erster technischer Vergleich und **kein statistisch abgesicherter Performance-Test**.

11.2 Messergebnisse

Nr.	Browser	Headless	Reuse	Suite-Laufzeit	Neue Prozesse nach Abschluss
1	Edge	true	false	23,45 s	20
2	Edge	true	true	26,35 s	8
3	Edge	false	false	35,27 s	18
4	Edge	false	true	24,09 s	9
5	Chrome	true	false	28,66 s	11*
6	**Chrome**	**true**	**true**	**17,45 s**	**13**
7	Chrome	false	false	29,33 s	15
8	Chrome	false	true	18,25 s	12
9	Chromium	true	false	22,71 s	3
10	Chromium	true	true	46,96 s	1

11	Chromium	false	false	33,51 s	14
12	Chromium	false	true	71,36 s	6

- **Datenabweichung:** Die manuelle Ergebnistabelle enthält für `chrome-true-false` den Wert **7**, der mitgelieferte CMD-Benchmark-Log dagegen **11**. Für diese Übersicht wurde der protokollierte Wert 11 verwendet; der Original-CSV-Eintrag sollte bei Bedarf gegengeprüft werden.

11.3 Interpretation

- **Schnellste Einzelmessung:** Chrome mit `headless: true` und `reuse: true` (**17,45 s**). Knapp dahinter: Chrome sichtbar mit Reuse (**18,25 s**).
- **Chrome profitiert in diesen Messungen von Reuse:** Ohne Reuse dauerte die Suite 28,66 bzw. 29,33 s, mit Reuse 17,45 bzw. 18,25 s.
- **Edge ist nicht der einzige betroffene Browser:** Auch Chrome und Chromium hinterließen nach der Suite noch Prozesse.
- **Chromium mit Reuse war auffällig langsam:** 46,96 s im Headless- und 71,36 s im sichtbaren Modus. Warum, lässt sich mit je einer Messung und ohne Detail-Logs nicht sicher bestimmen.
- **Wenige verbleibende Prozesse sind kein Performance-Sieg:** Chromium headless/reuse hatte zwar nur einen neu sichtbaren Prozess, war aber deutlich langsamer als Chrome. Keiner der zwölf Fälle erreichte null.

Vorläufige Empfehlung: Für schnelle lokale Regressionen bzw. einen weiteren Performance-Test ist **Chrome,** `headless: true` **,** `reuse: true` der sinnvollste Kandidat. Das ist **keine Freigabe für unbeaufsichtigten Dauerbetrieb**, solange das Prozess-Cleanup ungeklärt ist. Für reproduzierbare Vergleiche die Tests mehrfach ausführen und Median bzw. Streuung vergleichen.

11.4 Grenzen der Prozessmessung

Die Benchmark-Hilfe erfasst vor und unmittelbar nach der Suite Prozesslisten und zählt neue noch laufende Browser-PIDs. Das sind **Momentaufnahmen**: Manche Prozesse können später selbst enden, manche können länger bestehen bleiben. Zudem muss die Prozesszuordnung über den tatsächlich gestarteten Browser und dessen Profil erfolgen. Die Messung beweist deshalb einen Cleanup-Befund zum Prüfzeitpunkt, nicht die dauerhafte Lebensdauer jedes einzelnen Prozesses.

Für eine belastbare Wiederholung sollten die Prozesszahlen direkt am Suite-Ende sowie nach z. B. 5 und 30 Sekunden protokolliert werden. Die relevanten Logs und PID-Listen liegen im Ordner `benchmark-logs`.

12. Offene Sessions und bekannte Einschränkungen

12.1 Beabsichtigt offene Session vs. zurückgebliebener Prozess

Während eines TestCase darf die GUI-Session offen bleiben, damit mehrere GUI-Schritte denselben Browserzustand verwenden können. Bei `reuse: true` darf auch **der Browserprozess** bis zum Ende der Suite weiterlaufen – obwohl zwischen TestCases neue Contexts angelegt werden. Beides ist **beabsichtigtes Verhalten** und kein Leak.

Ein **Prozess-Leak** liegt dagegen vor, wenn nach Abschluss der gesamten Suite und ihres Cleanups browserbezogene Prozesse weiterlaufen, die für keinen aktiven Test mehr benötigt werden. Ein Windows-Browserprozess ist dabei nicht mit einer fachlich angemeldeten Web-Session gleichzusetzen: Ob serverseitige Logins oder Anwendungs-Sessions beendet sind, wurde durch den Browserbenchmark **nicht** geprüft.

12.2 Technischer Status

In der 1.0.72-basierten Implementierung wurde `Playwright.close()` unter Windows wiederholt erst nach dem Timeout durch einen Fallback beendet (`DRIVER_FORCE_STOP`). Die interne Process-Tree-Prüfung konnte `childProcessLeaks=0` melden, während die unabhängige Windows-Prozessabfrage noch Playwright-Browserprozesse fand. Deshalb ist `cleanupFailures=0` **kein Beweis für vollständige Prozessbereinigung**.

Die später untersuchten Profil-Prüfvarianten 1.0.73/1.0.74 lieferten auf dem untersuchten Windows-Rechner keine verlässliche Lösung und sind **nicht** die Referenz für diese Dokumentation. Version 1.0.75 baut funktional auf 1.0.72 auf und reduziert lediglich die Logmenge.

12.3 Auswirkungen und Empfehlungen für den Betrieb

- Für die tägliche Testausführung den kompakten Standardlog verwenden; `lifecycleLog: true` bei Störungen einschalten.
- `reuse: true` spart je nach Browser und Laufzeit Arbeit durch weniger Browserstarts, ohne die TestCase-Isolation aufzugeben.
- GUI-Testfälle so schreiben, dass sie einzeln oder als Teil einer Suite laufen können.
- Bei parallelen Suiten verschiedene Umgebungen und **externe** Testressourcen (Kartenleser, Karten, Datenbankzustände) getrennt betrachten.
- Browserprozesse des persönlichen Benutzerprofils niemals pauschal mit `taskkill /IM msedge.exe` oder entsprechend für Chrome beenden.
- Das Prozess-Leak separat untersuchen, statt es durch höhere GUI-Timeouts oder noch mehr normale Logausgabe zu kaschieren.

Fazit: Die Browserwahl, isolierte Contexts, Browser-Reuse und die GUI-Assertions sind funktional erprobt. Die schnellste gemessene Variante war **Chrome + Headless + Reuse**. Die vollständige und saubere Freigabe des Windows-Prozess-Cleanups bleibt ein **offener technischer Punkt**.